

BeatSync

Admin Panel Implementation Plan

MVP scope · Next.js · Firebase · Two roles: CEO + Trainer

Build order — application first

The mobile app is the product. Without it, the admin panel manages nothing. Wire the Flutter prototype to Firebase first, then build the admin panels on top of the same backend.

#	Phase	Duration	Why this order
1	Wire Flutter prototype to Firebase (auth · Firestore · BLE · workout save)	3-4 days	Prototype is half-done — fastest path to a working product
2	Trainer admin panel (Next.js)	2-3 days	Once trainers exist in Firebase, give them a desktop tool to manage
3	CEO admin panel (Next.js)	1-2 days	Once a few studios are signed up, you need platform-wide visibility

Clincher: you can manually onboard your first paying trainer via the Firebase console (add their studio doc by hand). They can use the mobile app to run their classes immediately. They don't strictly need the admin panel for week 1. They **do** need the mobile app.

Architecture

One Next.js codebase. Two route trees:

```
admin.beatsync.app/  
■■■ /login ← shared  
■■■ /studio/* ← Trainer panel (role: trainer)  
■ ■■■ /studio dashboard  
■ ■■■ /studio/members  
■ ■■■ /studio/sessions  
■ ■■■ /studio/billing  
■ ■■■ /studio/settings  
■■■ /admin/* ← CEO panel (role: ceo)  
■■■ /admin dashboard  
■■■ /admin/studios  
■■■ /admin/users  
■■■ /admin/billing  
■■■ /admin/team  
■■■ /admin/audit
```

Why one codebase: shared auth, shared design system, shared component library, shared Firebase SDK setup. Saves a solo dev ~30% of the work. Auth claims (role, studioId) decide which routes

the user can access.

Tech stack inside Next.js

Layer	Choice	Why
Framework	Next.js 15 (App Router)	Server components for fast loads, route handlers for Stripe webhooks later
Language	TypeScript	Auth claims + Firestore types are unreadable without it
Styling	Tailwind CSS	Matches BeatSync design tokens fast
Components	shadcn/ui	Copy-paste, accessible, customizable
Tables	TanStack Table v8	Industry-standard data grid
Forms	React Hook Form + Zod	Type-safe form validation
State	TanStack Query	Cache Firestore data, dedupe requests
Backend	Firebase JS SDK + Admin SDK	Same backend as the Flutter app
Charts	Recharts	Simple, clean defaults
CSV exports	papaparse	One library call
Icons	Lucide React	Matches the design language

Phase A — Foundation

Duration: 1 day · Goal: clickable shell with auth, no real data yet

- Init Next.js project, Tailwind, shadcn/ui base components
- Port BeatSync design tokens (brand red, dark theme, zone colors) into Tailwind config
- Firebase JS SDK config + auth provider context
- Auth pages: /login with brand red logo + email/password — same visual language as Flutter login
- Role-based route guard middleware: reads custom claims from ID token
- Trainer layout: sidebar (Dashboard / Members / Sessions / Billing / Settings) + top bar
- CEO layout: sidebar (Dashboard / Studios / Users / Billing / Team / Audit / Config) + top bar
- Shared components scaffold: DataTable, PageShell, EmptyState, LoadingSkeleton, ConfirmDialog, Toast

Deliverable: log in as trainer or CEO and land on the correct empty dashboard.

Phase B — Trainer Panel

Duration: 1.5-2 days · Goal: trainer can run their studio from desktop

1. **Dashboard** — this week's session count, active members, upcoming sessions list, quick CTAs
2. **Members table** — search, filter, sortable columns (name, email, attendance, last seen, joined date)
3. **Member detail page** — profile, attendance heatmap, fitness trend chart, notes field, action menu (remove, message)
4. **Invite members** — modal with email invite, shareable link, QR code download, CSV bulk upload
5. **Sessions list** — past sessions table with date, type, athlete count, avg TRIMP
6. **Session detail** — per-athlete breakdown (BPM, zone time, position), export as PDF
7. **Session scheduler** — calendar view + create-session form (one-off + recurring)
8. **Studio settings** — studio name, member capacity, invite code regenerate, TV view URL with QR
9. **Billing** — scaffolded only for MVP — shows current tier + member usage bar. Stripe wiring later.
10. **Account settings** — email change, password change, sign out

Deliverable: functional trainer admin panel with real Firestore data. Ship to your first paying studio.

Phase C — CEO Panel

Duration: 1-1.5 days · Goal: you and your brothers can monitor + support the platform

1. **Dashboard** — KPIs: total active studios + trend, total athletes, sessions this week, recent signups
2. **Studios table** — all studios with status, plan, member count, last active. Search + filter.
3. **Studio detail** — read-only mirror of trainer view. Actions: suspend / restore / comp / add note
4. **Users table** — all users across studios. Role filter (trainer/athlete), studio filter, status filter
5. **User detail** — profile, workouts, devices, login history. Actions: ban, restore, password reset, impersonate
6. **Team management** — invite a CEO (you + brothers), remove access. All CEOs have equal permissions.
7. **Audit log** — every destructive action recorded. Filterable by CEO, action type, date
8. **Configuration** — global feature flags, maintenance mode, minimum app version

Skip for MVP: platform analytics, communications, deep billing tools. Add post-launch.

Deliverable: you can sit at your laptop and understand what's happening across every studio.

Phase D — Backend Tie-in

Runs alongside Phase B + C — admin panels are useless without it

Data model

- Firestore collections finalized (users, studios, sessions, workouts, audit_log)
- Indexes for the filterable tables
- TypeScript types generated from Firestore schema

Authentication & permissions

- Firebase Auth custom claims via Admin SDK: { role: 'athlete' | 'trainer' | 'ceo', studioId? }
- Athletes read/write own user doc + workouts only
- Trainers read/write their own studio + its members + its sessions
- CEOs read everything; writes go through Cloud Functions only (auditable)
- Only two admin-panel roles: trainer (per studio) and ceo (you + brothers, equal access)

Cloud Functions

- inviteAthleteToStudio — generates invite code, validates trainer
- suspendStudio — bulk-disables a studio (CEO only)
- impersonateUser — generates short-lived token (CEO only)
- logAdminAction — every destructive action goes through this for the audit log

Deliverable: the admin panel actually does things, securely.

Realistic timeline

Day	Work
Day 1	Phase A (foundation) + start Phase D (data model)
Day 2	Phase B parts 1–5 (dashboard, members, invites, sessions)
Day 3	Phase B parts 6–10 (session detail, scheduler, settings, billing scaffold, account)
Day 4	Phase C parts 1–5 (CEO dashboard, studios, users, impersonate)
Day 5	Phase C parts 6–8 (team, audit, config) + Phase D wrap-up (security rules, indexes)

~5 working days for both panels to a launchable MVP — meaning you can sign up a paying trainer and run them through their studio panel without embarrassment.

What ships as the MVP launch

Trainer panel: 100% of Phase B (all ten features above)

CEO panel: dashboard + studios + users + team + audit (skip analytics, communications, deep billing for MVP)

Everything else (communications, deep analytics, Stripe wiring, exports) is post-launch — added once you have a paying studio asking for it.

BeatSync · Solo developer · MVP build phase